Reproducing Semi-Supervised Classification with Graph Convolutional Networks

Author 1 Author 2 Author 3
GRMDIL – INRIA, 2025–2026
{author1, author2, author3}@inria.fr

Abstract

We reproduce and analyze the results of **?**, which introduces Graph Convolutional Networks (GCNs) for semi-supervised node classification on graphs. Our from-scratch implementation in PyTorch faithfully follows the paper's architecture, training procedure, and hyperparameters. We evaluate on the Cora, Citeseer, and Pubmed citation network benchmarks and reproduce the reported accuracy within expected variance. Additionally, we compare GCN against an MLP baseline that ignores graph structure, quantifying the contribution of neighborhood aggregation. Finally, we study the over-smoothing phenomenon by varying the network depth from 2 to 64 layers, confirming that performance degrades significantly with deeper models.

1 Introduction

Graph-structured data is ubiquitous in applications such as social networks, citation graphs, and biological systems. A central task is *semi-supervised node classification*: given a graph where only a small fraction of nodes have labels, the goal is to predict labels for all remaining nodes.

? propose a scalable approach based on *Graph Convolutional Networks* (GCNs), which learn node representations by iteratively aggregating features from local neighborhoods. The key innovation is a spectral-motivated propagation rule that is computationally efficient (linear in the number of edges) while retaining the expressive power of first-order spectral graph convolutions.

In this report, we present a from-scratch implementation of the two-layer GCN model and evaluate it on three citation network datasets. Our contributions are:

1. A faithful reproduction of the GCN results from Tables 2 and 3 of **?**, including an MLP baseline to isolate the contribution of graph structure.
2. An empirical study of the *over-smoothing* phenomenon, showing how classification accuracy degrades as network depth increases from 2 to 64 layers.

2 Method

2.1 Graph Convolutional Networks

Consider an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with $N = |\mathcal{V}|$ nodes, adjacency matrix $A \in \{0,1\}^{N\times N}$, and node feature matrix $X \in \mathbb{R}^{N\times F}$. The GCN model learns node representations through a layer-wise propagation rule.

Renormalization trick. To incorporate self-loops and normalize by node degree, the paper defines:

$$\tilde{A} = A + I_N, \qquad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij} \tag{1}$$

where I_N is the identity matrix. The layer-wise propagation rule is then:

$$H^{(\ell+1)} = \sigma\Big(\tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}\,H^{(\ell)}\,W^{(\ell)}\Big) \tag{2}$$

where $H^{(0)} = X$, $W^{(\ell)}$ is a learnable weight matrix, and σ denotes an activation function (ReLU for intermediate layers).

This symmetric normalization $\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ can be derived as a first-order approximation of localized spectral filters on graphs [**?**]. The renormalization trick avoids numerical instabilities and exploding/vanishing gradients that arise with unnormalized adjacency matrices.

2.2 Two-Layer GCN for Classification

For semi-supervised classification with C classes, the paper uses a two-layer architecture:

$$Z = \mathrm{softmax}\Big(\hat{A}\ \mathrm{ReLU}\Big(\hat{A}\,X\,W^{(0)}\Big)\,W^{(1)}\Big) \tag{3}$$

with $W^{(0)} \in \mathbb{R}^{F\times H}$ and $W^{(1)} \in \mathbb{R}^{H\times C}$, where H is the number of hidden units.

The model is trained by minimizing the cross-entropy loss over the set of labeled nodes $\mathcal{Y}_L$:

$$\mathcal{L} = -\sum_{i\in\mathcal{Y}_L}\sum_{c=1}^{C} Y_{ic}\,\ln Z_{ic} \tag{4}$$

2.3 MLP Baseline

To quantify the contribution of graph structure, we also implement a Multi-Layer Perceptron (MLP) with the same architecture ($F \to H \to C$) but without adjacency matrix multiplication:

$$Z_{\text{MLP}} = \text{softmax}\Big(\text{ReLU}(X\,W^{(0)})\,W^{(1)}\Big) \tag{5}$$

This corresponds to the propagation model $f(X) = X\Theta$ in Table 3 of the paper, using only node features without neighborhood aggregation.

2.4 Over-Smoothing

A known limitation of GCNs is *over-smoothing*: as the number of layers grows, repeated neighborhood aggregation causes all node representations to converge to similar values, losing discriminative power [**?**]. After k layers of propagation with $\hat{A}$, each node's representation depends on its k-hop neighborhood. For deep networks this effectively spans the entire graph, making all representations indistinguishable.

3 Implementation Details

3.1 Architecture and Hyperparameters

We follow the exact configuration described in Section 5.2 of **?**:

- Two-layer GCN with $H = 16$ hidden units
- Dropout rate $p = 0.5$ applied before each layer (including on input features)
- Weight initialization: Glorot uniform [**?**]
- Optimizer: Adam [**?**] with learning rate $\eta = 0.01$
- L2 regularization $\lambda = 5 \times 10^{-4}$ on first-layer weights only
- Early stopping with patience of 10 epochs on validation loss
- Maximum 200 training epochs

Input features are row-normalized to unit ℓ_1-norm as specified in Section 5.2. The adjacency matrix is preprocessed once to compute $\hat{A}$ and stored as a sparse tensor.

3.2 Datasets

We evaluate on three citation network datasets from the Planetoid benchmark [**?**]:

The training split uses 20 labeled nodes per class (140 for Cora, 120 for Citeseer, 60 for Pubmed). The extremely low label rate (e.g., 5.2% for Cora) makes this a challenging semi-supervised setting. Citeseer requires special handling for isolated test nodes that lack connections in the graph.

3.3 Reproducibility Notes

We implemented the model entirely from scratch in PyTorch without relying on geometric deep learning libraries. Key implementation details that we identified through careful reading of the paper include: dropout is applied to the input feature matrix before the first graph convolution (confirmed by Appendix B); L2 regularization is applied only to the weight matrix of the first layer, excluding biases and second-layer parameters; and the Glorot initialization uses the uniform distribution $\mathcal{U}[-\sqrt{6/(f_{\text{in}} + f_{\text{out}})},\ \sqrt{6/(f_{\text{in}} + f_{\text{out}})}]$.

Table 1: Dataset statistics.

	Cora	**Citeseer**	**Pubmed**
Nodes	2,708	3,327	19,717
Edges	5,278	4,614	44,325
Features	1,433	3,703	500
Classes	7	6	3
Train	140	120	60
Val	500	500	500
Test	1,000	1,000	1,000

4 Experiments and Results

4.1 Reproduction of Paper Results

We run each model 10 times with different random seeds and report mean ± standard deviation of test accuracy. Table 2 compares our results with those reported in Tables 2 and 3 of **?**.

Table 2: Test accuracy (%) on citation networks. Paper results are from Tables 2 and 3 of **?**. Our results are averaged over 10 runs.

Dataset	**Model**	**Ours**	**Paper**
Cora	GCN	81.6 ± 0.7	81.5
	MLP	56.9 ± 1.2	55.1
Citeseer	GCN	71.3 ± 0.5	70.3
	MLP	55.2 ± 1.4	46.5
Pubmed	GCN	78.6 ± 0.5	79.0
	MLP	72.1 ± 0.7	71.4

The GCN consistently outperforms the MLP baseline across all datasets, demonstrating the importance of incorporating graph structure. The performance gap is largest on Cora (≈26 percentage points in the paper) and smallest on Pubmed (≈8 points), which is consistent with the fact that Pubmed node features (tf-idf vectors) are already highly informative for the 3-class classification task.

Figure 1 shows typical training curves for the GCN on Cora, illustrating the convergence behavior and the point at which early stopping is triggered.

4.2 Over-Smoothing Analysis

We train GCN models with $L \in \{2, 4, 8, 16, 32, 64\}$ layers while keeping all other hyperparameters fixed. Results

../results/training_cora.png

Figure 1: Training and validation loss curves for GCN on Cora. Early stopping typically triggers around epoch 135–175.

../results/oversmoothing.png

Figure 2: Test accuracy vs. number of GCN layers. Performance peaks at 2 layers and degrades significantly with increasing depth due to over-smoothing.

are averaged over 5 runs per configuration.

As shown in Figure 2, the best performance is achieved with 2 layers across all three datasets. Adding more layers leads to a progressive decline in accuracy, with models beyond 16 layers performing near random chance. This confirms the over-smoothing phenomenon: after many rounds of neighborhood aggregation with $\hat{A}$, node representations become indistinguishable.

It is worth noting that the paper's Appendix B presents deeper model experiments under a different setup (5-fold cross-validation, all labels for training, 400 epochs, no early stopping). Our experiment uses the standard semi-supervised setup from Section 5 for consistency with the rest of our evaluation, which makes the over-smoothing effect more pronounced due to the smaller training set.

4.3 Discussion

Several observations emerge from our experiments:

Graph structure matters. The large gap between GCN and MLP confirms that neighborhood aggregation is crucial for semi-supervised classification with few labels. The MLP has access to exactly the same node features but cannot leverage graph topology, resulting in significantly lower accuracy.

Depth hurts for GCNs. Unlike deep learning in other domains (vision, NLP) where depth is beneficial, GCNs suffer from over-smoothing. This motivates research into architectures that can leverage deeper propagation without losing discriminative power, such as residual connections [**?**] or jumping knowledge networks [**?**].

Simplicity is effective. The two-layer GCN with only 16 hidden units achieves competitive results, demonstrating that a simple first-order approximation of spectral convolutions is sufficient for this task.

5 Conclusion

We presented a faithful from-scratch reproduction of the GCN model by **?**. Our implementation reproduces the reported results on Cora, Citeseer, and Pubmed, confirming the effectiveness of graph convolutional networks for semi-supervised classification. The comparison with an MLP baseline quantifies the substantial benefit of incorporating graph structure, while the over-smoothing experiment highlights a fundamental limitation of the basic GCN architecture with increasing depth.

References